

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE CODE: ITA 0605

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO1: Learning Problems, Perspectives and Issues, Concept Learning, Version Spaces & Candidate Elimination, Inductive Bias, Decision Tree Learning, Representation & Heuristic Search (BL1, BL2)

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks: 25

Annexure A

APPLICATION BASED QUESTIONS

Code of Conduct:

I R NAVEEN KISHORE(192424261) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

6. A machine learning model shows the following loss values during training: (5 Marks)

(a) Epoch 1: Training Loss = 3.0, Validation Loss = 2.7

(b) Epoch 2: Training Loss = 2.6, Validation Loss = 2.3

(c) Epoch 3: Training Loss = 2.3, Validation Loss = 2.1

(d) Epoch 4: Training Loss = 2.1, Validation Loss = 2.0

(e) Epoch 5: Training Loss = 1.9, Validation Loss = 2.2

(f) Epoch 6: Training Loss = 1.7, Validation Loss = 2.5

(g) Epoch 7: Training Loss = 1.6, Validation Loss = 2.8

(h) Epoch 8: Training Loss = 1.4, Validation Loss = 3.1

i) At which epoch does the model achieve optimal generalization?

ii) Identify signs of overfitting from the data.

iii) Recommend two regularization techniques and explain their impact.

Program:

```
training_loss = [3.0, 2.6, 2.3, 2.1, 1.9, 1.7, 1.6, 1.4]
```

```
validation_loss = [2.7, 2.3, 2.1, 2.0, 2.2, 2.5, 2.8, 3.1]
```

```
# Find epoch with minimum validation loss
```

```
optimal_epoch = validation_loss.index(min(validation_loss)) + 1
```

```
print("Q6")
```

```
print("Optimal Generalization Epoch:", optimal_epoch)
```

```
print("Minimum Validation Loss:", min(validation_loss))
```

```
print("\nSigns of Overfitting:")
```

```
print("After Epoch 4, training loss continues to decrease,")
```

```
print("but validation loss starts increasing. This indicates overfitting.")
```

```
print("\nRegularization Techniques:")
```

```
print("1. L1/L2 Regularization - Penalizes large model weights and reduces complexity.")
```

```
print("2. Dropout - Randomly disables neurons during training to prevent over-dependence.")
```

```
def calculate_metrics(TP, FN, TN, FP):
```

```
    total = TP + FN + TN + FP
```

```
    accuracy = (TP + TN) / total
```

```
    precision = TP / (TP + FP)
```

```
    sensitivity = TP / (TP + FN)
```

```
    specificity = TN / (TN + FP)
```

```

f1_score = 2 * precision * sensitivity / (precision + sensitivity)
print("Accuracy :", round(accuracy, 4))
print("Precision :", round(precision, 4))
print("Sensitivity:", round(sensitivity, 4))
print("Specificity:", round(specificity, 4))
print("F1-Score :", round(f1_score, 4))

```

Output:

Q6

```

Optimal Generalization Epoch: 4
Minimum Validation Loss: 2.0

```

Signs of Overfitting:

After Epoch 4, training loss continues to decrease,
but validation loss starts increasing. This indicates overfitting.

Regularization Techniques:

1. L1/L2 Regularization - Penalizes large model weights and reduces complexity.
2. Dropout - Randomly disables neurons during training to prevent over-dependence.

7. In an industrial process, an AI robot is assigned to segregate the front and back tires automatically. Let the number of tires are 250 (125 front and 125 back tires). (5 Marks)

a. Front identified as Front: 119

Front identifies as Back: 06

Back identified as Back: 120

a. Back identified as Front: 05, Compute: Accuracy, Precision, Sensitivity, Specificity and F1-Score.

Program:

```

training_loss = [3.0, 2.6, 2.3, 2.1, 1.9, 1.7, 1.6, 1.4]
validation_loss = [2.7, 2.3, 2.1, 2.0, 2.2, 2.5, 2.8, 3.1]

```

```

# Find epoch with minimum validation loss
optimal_epoch = validation_loss.index(min(validation_loss)) + 1

```

```

print("Q6")
print("Optimal Generalization Epoch:", optimal_epoch)
print("Minimum Validation Loss:", min(validation_loss))

```

```

print("\nSigns of Overfitting:")
print("After Epoch 4, training loss continues to decrease,")
print("but validation loss starts increasing. This indicates overfitting.")

```

```

print("\nRegularization Techniques:")
print("1. L1/L2 Regularization - Penalizes large model weights and reduces complexity.")
print("2. Dropout - Randomly disables neurons during training to prevent over-dependence.")
print("\nQ7 - AI Robot Tire Segregation")
calculate_metrics(

```

TP=119,
FN=6,
TN=120,
FP=5

)

Output:

```
Q7 - AI Robot Tire Segregation
Accuracy   : 0.956
Precision   : 0.9597
Sensitivity: 0.952
Specificity: 0.96
F1-Score    : 0.9558
```

8. In a medical diagnosis system, an AI model detects whether a patient has a disease or not. Out of 300 cases (150 diseased and 150 healthy):

(5 Marks)

a. Diseased identified as Diseased: 138

a. Diseased identified as Healthy: 12

Healthy identified as Healthy: 135

a. Healthy identified as Diseased: 15

Compute: Accuracy, Precision, Sensitivity (Recall), Specificity, and F1-Score.

Program:

```
training_loss = [3.0, 2.6, 2.3, 2.1, 1.9, 1.7, 1.6, 1.4]
validation_loss = [2.7, 2.3, 2.1, 2.0, 2.2, 2.5, 2.8, 3.1]
optimal_epoch = validation_loss.index(min(validation_loss)) + 1
print("Q6")
print("Optimal Generalization Epoch:", optimal_epoch)
print("Minimum Validation Loss:", min(validation_loss))
print("\nSigns of Overfitting:")
```

```
print("After Epoch 4, training loss continues to decrease,")
print("but validation loss starts increasing. This indicates overfitting.")
```

```
print("\nRegularization Techniques:")
print("1. L1/L2 Regularization - Penalizes large model weights and reduces complexity.")
```

```
print("2. Dropout - Randomly disables neurons during training to prevent over-dependence.")
print("\nQ8 - Medical Diagnosis System")
```

```
calculate_metrics(
    TP=138,
```

FN=12,
TN=135,
FP=15
)

Output:

```
Q8 - Medical Diagnosis System
Accuracy   : 0.91
Precision   : 0.902
Sensitivity: 0.92
Specificity: 0.9
F1-Score    : 0.9109
```

9. In a spam email filtering system, an AI model classifies emails as Spam or Not Spam. Out of 400 emails (200 spam and 200 non-spam):

(5 Marks)

Spam identified as Spam: 180

Spam identified as Not Spam: 20

Not Spam identified as Not Spam: 170

Not Spam identified as Spam: 30

Compute: Accuracy, Precision, Sensitivity, Specificity, and F1-Score.

Program:

```
training_loss = [3.0, 2.6, 2.3, 2.1, 1.9, 1.7, 1.6, 1.4]
validation_loss = [2.7, 2.3, 2.1, 2.0, 2.2, 2.5, 2.8, 3.1]
```

```
# Find epoch with minimum validation loss
optimal_epoch = validation_loss.index(min(validation_loss)) + 1
```

```
print("Q6")
print("Optimal Generalization Epoch:", optimal_epoch)
print("Minimum Validation Loss:", min(validation_loss))
```

```
print("\nSigns of Overfitting:")
print("After Epoch 4, training loss continues to decrease,")
print("but validation loss starts increasing. This indicates overfitting.")
```

```
print("\nRegularization Techniques:")
print("1. L1/L2 Regularization - Penalizes large model weights and reduces complexity.")
print("2. Dropout - Randomly disables neurons during training to prevent over-dependence.")
print("\nQ9 - Spam Email Filtering")
calculate_metrics(
    TP=180,
    FN=20,
```

TN=170,
FP=30
)

Output:

```
Q9 - Spam Email Filtering
Accuracy   : 0.875
Precision  : 0.8571
Sensitivity: 0.9
Specificity: 0.85
F1-Score   : 0.878
```

10. In a quality inspection system in manufacturing, an AI model identifies defective and non-defective products. Out of 500 products (250 defective and 250 non-defective):

Defective identified as Defective: 230

(5 Marks)

Defective identified as Non-Defective: 20

Non-Defective identified as Non-Defective: 225

Non-Defective identified as Defective: 25

Compute: Accuracy, Precision, Sensitivity, Specificity, and F1-Score.

Program:

```
training_loss = [3.0, 2.6, 2.3, 2.1, 1.9, 1.7, 1.6, 1.4]
validation_loss = [2.7, 2.3, 2.1, 2.0, 2.2, 2.5, 2.8, 3.1]
```

```
# Find epoch with minimum validation loss
optimal_epoch = validation_loss.index(min(validation_loss)) + 1
```

```
print("Q6")
print("Optimal Generalization Epoch:", optimal_epoch)
print("Minimum Validation Loss:", min(validation_loss))
```

```
print("\nSigns of Overfitting:")
print("After Epoch 4, training loss continues to decrease,")
print("but validation loss starts increasing. This indicates overfitting.")
```

```
print("\nRegularization Techniques:")
print("1. L1/L2 Regularization - Penalizes large model weights and reduces complexity.")
print("2. Dropout - Randomly disables neurons during training to prevent over-dependence.")
print("\nQ10 - Quality Inspection System")
calculate_metrics(
    TP=230,
    FN=20,
    TN=225,
```

FP=25

)

Output:

```
Q10 - Quality Inspection System
Accuracy   : 0.91
Precision  : 0.902
Sensitivity: 0.92
Specificity: 0.9
F1-Score   : 0.9109
```